```
from google.colab import files

uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving validation.csv to validation.csv
Saving train.csv to train.csv
Saving test.csv to test.csv
```

```
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    f1_score
)

train_df = pd.read_csv("train.csv")
val_df = pd.read_csv("validation.csv")
test_df = pd.read_csv("test.csv")

# Remove rows with missing text or labels
train_df = train_df.dropna(subset=["Headline", "Language"])
val_df = val_df.dropna(subset=["Headline", "Language"])
test_df = test_df.dropna(subset=["Headline", "Language"])

# Standardize text fields
for df in [train_df, val_df, test_df]:
    df["Headline"] = df["Headline"].astype(str).str.strip()
    df["Language"] = df["Language"].astype(str).str.strip()

    df.drop(
        df[df["Headline"] == ""].index,
        inplace=True
    )

print("Training rows:", len(train_df))
print("Validation rows:", len(val_df))
print("Test rows:", len(test_df))
print("Languages:", train_df["Language"].nunique())
print(sorted(train_df["Language"].unique()))

display(train_df.head())
```

```
Training rows: 95045
Validation rows: 20368
Test rows: 20368
Languages: 18
['Assamese', 'Bengali', 'English', 'Gujarati', 'Hindi', 'Kannada', 'Kashmiri', 'Konkani', 'Malayalam', 'Marathi', 'Nepali',
```

	Headline	Language
0	ଓଡ଼ିଶାରେ ମାଦ୍ୱିସ ଟାୟାର୍ସର ପ୍ରଥମ ବିତରକ ନିଯୁକ୍ତ	Odia
1	घडीका पुर्जामा पनि सुन तस्करी	Nepali
2	श्रद्धा कपूर को महादेव बेटिंग ऐप घोटाला में ईड...	Hindi
3	চিৰিয়াখানাত এটা চিহ্ন , কাতালান - স্পেনিছ্ - ই...	Assamese
4	नारायणं संनिरीक्ष्य प्रसन्नेनान्तरात्मना। नमस्...	Sanskrit

```
print("Training class distribution:")
display(train_df["Language"].value_counts().to_frame("Count"))

print("Missing values:")
print(train_df.isna().sum())

train_df["word_count"] = (
    train_df["Headline"].str.split().str.len()
)
```

```
print("\nHeadline length statistics:")
print(train_df["word_count"].describe())

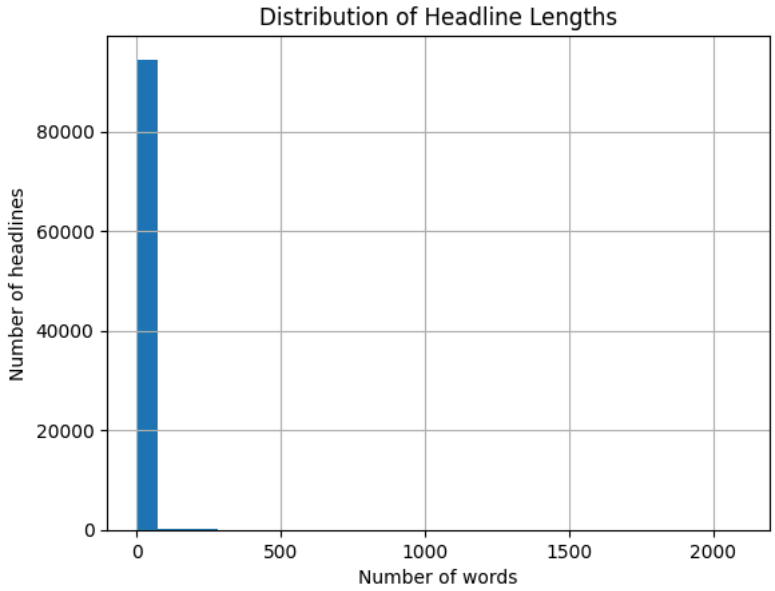
train_df["word_count"].hist(bins=30)
plt.title("Distribution of Headline Lengths")
plt.xlabel("Number of words")
plt.ylabel("Number of headlines")
plt.show()
```

```
Training class distribution:
```

Language	Count
Punjabi	10552
Odia	9981
Konkani	9869
Hindi	9428
Sanskrit	8252
Bengali	7025
English	6872
Sindhi	6186
Nepali	6086
Marathi	4637
Gujarati	2652
Telugu	2494
Malayalam	2396
Tamil	2236
Kannada	1855
Kashmiri	1597
Urdu	1590
Assamese	1337

```
Missing values:
Headline    0
Language    0
dtype: int64

Headline length statistics:
count    95045.000000
mean        12.341849
std         19.812009
min          1.000000
25%          8.000000
50%         11.000000
75%         14.000000
max       2092.000000
Name: word_count, dtype: float64
```



```
X_train = train_df["Headline"]
y_train = train_df["Language"]

X_val = val_df["Headline"]
y_val = val_df["Language"]

X_test = test_df["Headline"]
y_test = test_df["Language"]

model = Pipeline([
    ("tfidf", TfidfVectorizer(
        analyzer="char",
        ngram_range=(2, 5),
        max_features=100000,
        sublinear_tf=True,
        dtype=np.float32
    )),
    ("classifier", LogisticRegression(
        max_iter=300,
        random_state=42
    ))
])

model.fit(X_train, y_train)

print("BhashaLens training completed!")
```

```
BhashaLens training completed!
```

```
val_predictions = model.predict(X_val)

print("Validation accuracy:",
      round(accuracy_score(y_val, val_predictions), 4))

print("\nValidation macro F1:",
      round(f1_score(
          y_val, val_predictions, average="macro"
      ), 4))

test_predictions = model.predict(X_test)

print("\nFINAL TEST ACCURACY:",
      round(accuracy_score(y_test, test_predictions), 4))

print("\nFINAL TEST MACRO F1:",
      round(f1_score(
          y_test, test_predictions, average="macro"
      ), 4))

print("\nDetailed classification report:")
print(classification_report(
    y_test,
    test_predictions,
    digits=4,
    zero_division=0
))
```

```
Validation accuracy: 0.997

Validation macro F1: 0.997

FINAL TEST ACCURACY: 0.9973

FINAL TEST MACRO F1: 0.9968

Detailed classification report:
              precision    recall  f1-score   support

    Assamese     0.9964    0.9615    0.9786       286
     Bengali     0.9934    0.9993    0.9964      1505
     English     1.0000    1.0000    1.0000      1473
    Gujarati     1.0000    0.9930    0.9965       568
       Hindi     0.9863    0.9995    0.9929      2021
     Kannada     1.0000    0.9975    0.9987       398
    Kashmiri     1.0000    0.9942    0.9971       343
     Konkani     1.0000    1.0000    1.0000      2114
   Malayalam     1.0000    0.9961    0.9981       514
     Marathi     0.9890    0.9980    0.9935       994
      Nepali     0.9985    0.9985    0.9985      1304
        Odia     1.0000    1.0000    1.0000      2138
     Punjabi     0.9996    0.9881    0.9938      2261
    Sanskrit     0.9994    0.9994    0.9994      1768
      Sindhi     1.0000    0.9992    0.9996      1326
       Tamil     1.0000    1.0000    1.0000       479
```

```
          Telugu     1.0000    1.0000    1.0000       535
           Urdu      0.9971    1.0000    0.9985       341

        accuracy                         0.9973     20368
       macro avg     0.9978    0.9958    0.9968     20368
    weighted avg     0.9973    0.9973    0.9973     20368
```

```
labels = sorted(y_test.unique())

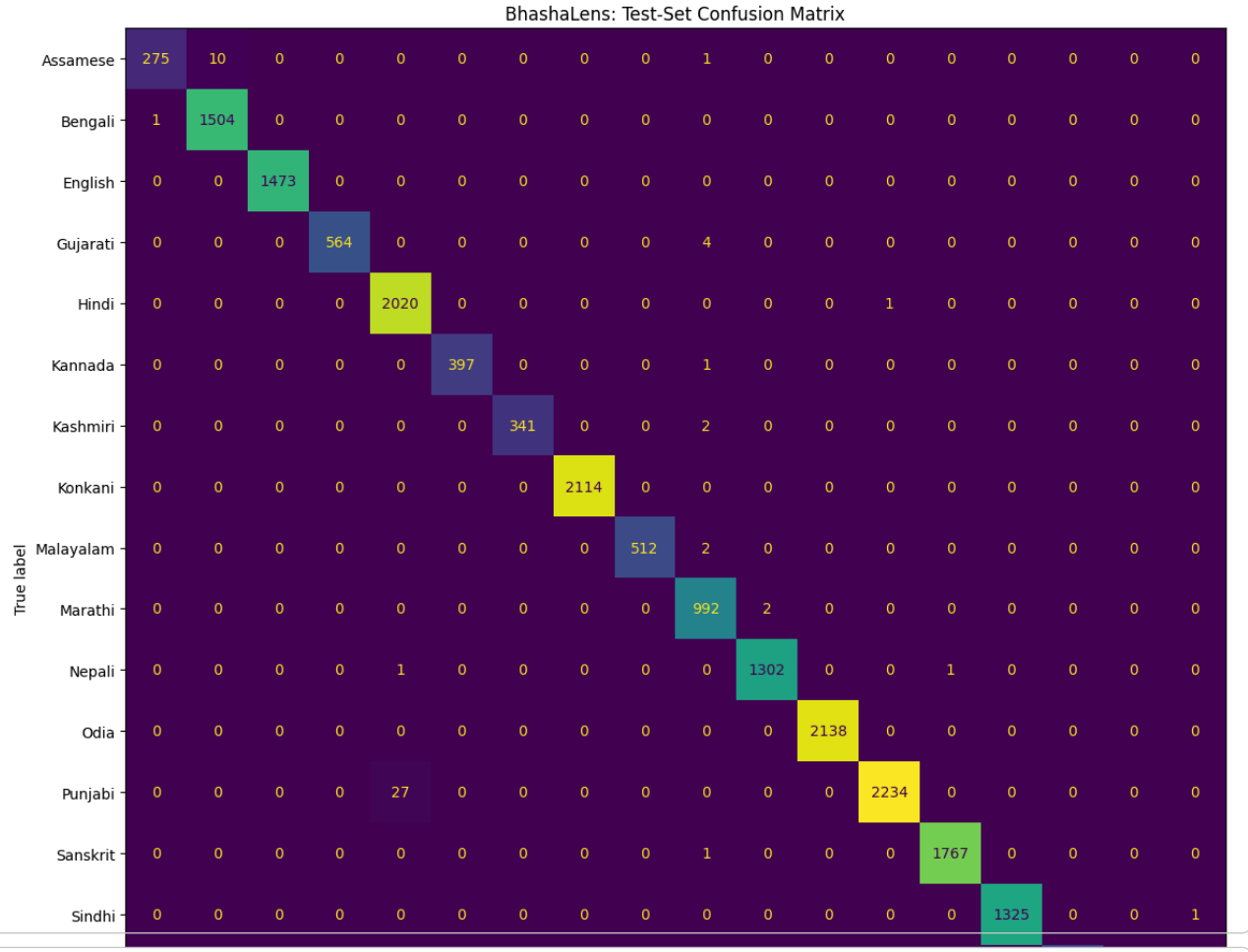
cm = confusion_matrix(
    y_test,
    test_predictions,
    labels=labels
)

fig, ax = plt.subplots(figsize=(14, 12))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=labels
)

disp.plot(
    ax=ax,
    xticks_rotation=90,
    colorbar=False
)

plt.title("BhashaLens: Test-Set Confusion Matrix")
plt.tight_layout()
plt.show()
```




```
# Determine the threshold from training data only
train_lengths = X_train.str.split().str.len()
threshold = train_lengths.median()

test_lengths = X_test.str.split().str.len()

length_groups = {
    "Short headlines": test_lengths <= threshold,
    "Long headlines": test_lengths > threshold
}
```

```
audit_results = []

for group_name, mask in length_groups.items():
    actual = y_test.loc[mask]
    predicted = test_predictions[mask.to_numpy()]

    audit_results.append({
        "Group": group_name,
        "Sample_Count": int(mask.sum()),
        "Accuracy": accuracy_score(actual, predicted),
        "Macro_F1": f1_score(
            actual, predicted, average="macro"
        )
    })

audit_df = pd.DataFrame(audit_results)

print("Training median word count:", threshold)
display(audit_df.round(4))

accuracy_gap = abs(
    audit_df.iloc[0]["Accuracy"]
    - audit_df.iloc[1]["Accuracy"]
) * 100

print(f"Absolute accuracy gap: {accuracy_gap:.2f} percentage points")
```

```
Training median word count: 11.0
```

	Group	Sample_Count	Accuracy	Macro_F1
0	Short headlines	11722	0.9978	0.9971
1	Long headlines	8646	0.9966	0.9989

```
Absolute accuracy gap: 0.11 percentage points
```

```
from sklearn.metrics import recall_score

language_results = []

for language in sorted(y_test.unique()):
    mask = (y_test == language).to_numpy()

    actual = y_test[mask]
    predicted = test_predictions[mask]

    language_results.append({
        "Language": language,
        "Sample_Count": int(mask.sum()),
        "Accuracy": accuracy_score(actual, predicted),
        "F1_Score": f1_score(
            actual,
            predicted,
            labels=[language],
            average="macro",
            zero_division=0
        ),
        "Recall": recall_score(
            actual,
            predicted,
            labels=[language],
            average="macro",
            zero_division=0
        )
    })

language_audit = pd.DataFrame(language_results)
language_audit = language_audit.sort_values("Recall")

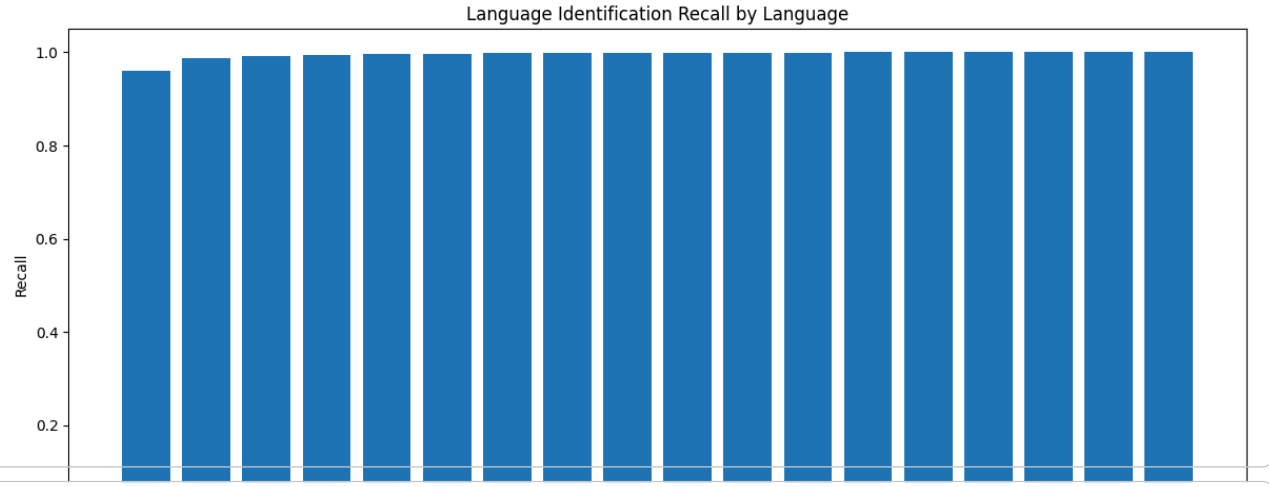
display(language_audit.round(4))

language_audit.to_csv(
    "language_audit_results.csv",
    index=False
)

plt.figure(figsize=(12, 6))
plt.bar(
    language_audit["Language"],
    language_audit["Recall"]
)
plt.xticks(rotation=60, ha="right")
plt.ylabel("Recall")
```

```
plt.title("Language Identification Recall by Language")
plt.tight_layout()
plt.show()
```

	Language	Sample_Count	Accuracy	F1_Score	Recall
0	Assamese	286	0.9615	0.9804	0.9615
12	Punjabi	2261	0.9881	0.9940	0.9881
3	Gujarati	568	0.9930	0.9965	0.9930
6	Kashmiri	343	0.9942	0.9971	0.9942
8	Malayalam	514	0.9961	0.9981	0.9961
5	Kannada	398	0.9975	0.9987	0.9975
9	Marathi	994	0.9980	0.9990	0.9980
10	Nepali	1304	0.9985	0.9992	0.9985
14	Sindhi	1326	0.9992	0.9996	0.9992
1	Bengali	1505	0.9993	0.9997	0.9993
13	Sanskrit	1768	0.9994	0.9997	0.9994
4	Hindi	2021	0.9995	0.9998	0.9995
7	Konkani	2114	1.0000	1.0000	1.0000
2	English	1473	1.0000	1.0000	1.0000
11	Odia	2138	1.0000	1.0000	1.0000
15	Tamil	479	1.0000	1.0000	1.0000
16	Telugu	535	1.0000	1.0000	1.0000
17	Urdu	341	1.0000	1.0000	1.0000




```
vectorizer = model.named_steps["tfidf"]
classifier = model.named_steps["classifier"]

features = vectorizer.get_feature_names_out()
classes = classifier.classes_
coefficients = classifier.coef_

for language in ["Hindi", "Marathi", "Bengali"]:
    if language not in classes:
        continue

    class_index = list(classes).index(language)
    top_indices = np.argsort(
        coefficients[class_index]
    )[-15:][::-1]

    print(f"\nTop character patterns for {language}:")
    print([
        (features[i], round(coefficients[class_index][i], 3))
        for i in top_indices
    ])
```

```
Top character patterns for Hindi:
[(' क', np.float64(6.698)), ('े ', np.float64(5.058)), (' के ', np.float64(4.049)), ('के ', np.float64(3.938)), ('र ', np.fl

Top character patterns for Marathi:
[('चा ', np.float64(5.346)), ('ी ', np.float64(4.347)), ('ला ', np.float64(4.253)), ('चा', np.float64(4.097)), ('ा ', np.fl
```

```
Top character patterns for Bengali:
[('ে ', np.float64(6.418)), ('র ', np.float64(6.342)), ('ার', np.float64(4.648)), ('ের', np.float64(3.999)), ('া', np.flo
```

```
def predict_language(text):
    prediction = model.predict([text])[0]

    print("Input:", text)
    print("Predicted language:", prediction)

examples = [
    "यह एक उदाहरण वाक्य है।",
    "This is an example sentence.",
    "এটি একটি উদাহরণ বাক্য।"
]

for example in examples:
    predict_language(example)
    print("-" * 40)

# Try any sentence you want:
my_text = input("Enter a headline or sentence: ")
predict_language(my_text)
```

```
Input: यह एक उदाहरण वाक्य है।
Predicted language: Hindi
----------------------------------------
Input: This is an example sentence.
Predicted language: English
----------------------------------------
Input: এটি একটি উদাহরণ বাক্য।
Predicted language: Bengali
----------------------------------------
Enter a headline or sentence: तपाई कस्तो हुनुहुन्छ
Input: तपाई कस्तो हुनुहुन्छ
Predicted language: Nepali
```

```
import json
import joblib
from sklearn.metrics import accuracy_score

results = {
    "project": "BhashaLens",
    "dataset": "Indian Language Identification",
    "model": "Character TF-IDF + Logistic Regression",
    "training_samples": int(len(X_train)),
    "validation_samples": int(len(X_val)),
    "test_samples": int(len(X_test)),
    "number_of_languages": int(len(classes)),
    "test_accuracy": float(
        accuracy_score(y_test, test_predictions)
    ),
    "test_macro_f1": float(
        f1_score(y_test, test_predictions, average="macro")
    ),
    "short_vs_long_accuracy_gap_percentage_points": float(
        accuracy_gap
    )
}

with open("bhashalens_results.json", "w") as f:
    json.dump(results, f, indent=4)

language_audit.to_csv(
    "language_audit_results.csv",
    index=False
)

audit_df.to_csv(
    "length_audit_results.csv",
    index=False
)

joblib.dump(model, "bhashalens_model.joblib")

print(json.dumps(results, indent=4))
print("All result files saved!")
```

```
{
    "project": "BhashaLens",
    "dataset": "Indian Language Identification",
    "model": "Character TF-IDF + Logistic Regression",
```

```
        "training_samples": 95045,
        "validation_samples": 20368,
        "test_samples": 20368,
        "number_of_languages": 18,
        "test_accuracy": 0.9972996857816182,
        "test_macro_f1": 0.9967518940152486,
        "short_vs_long_accuracy_gap_percentage_points": 0.11361006820709285
    }
    All result files saved!
```